

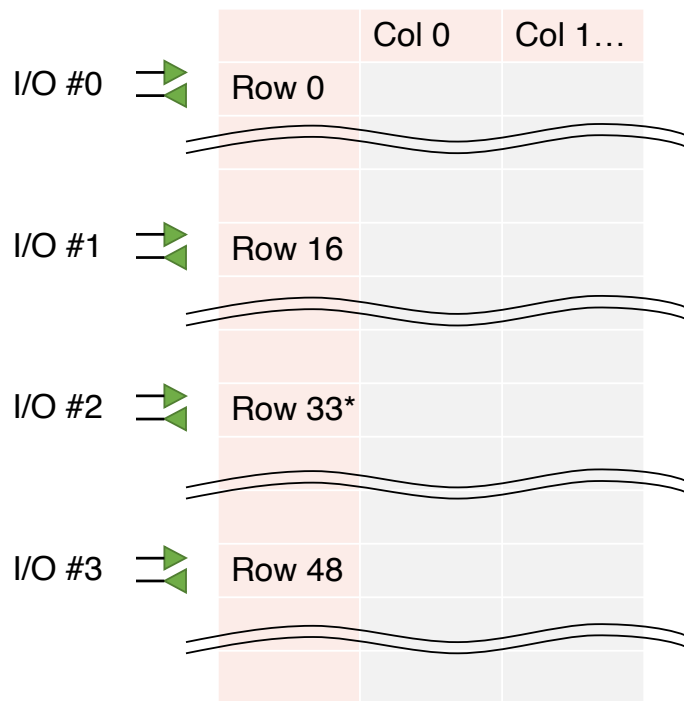
Host Runtime and Tensor Streaming

Contents

- [Enabling Tensor Streaming](#)
- [Using Streaming Mode](#)
- [Using Copy Mode](#)
- [Launching Kernels](#)
- [Using Buffers](#)
- [SdkRuntime Host API](#)

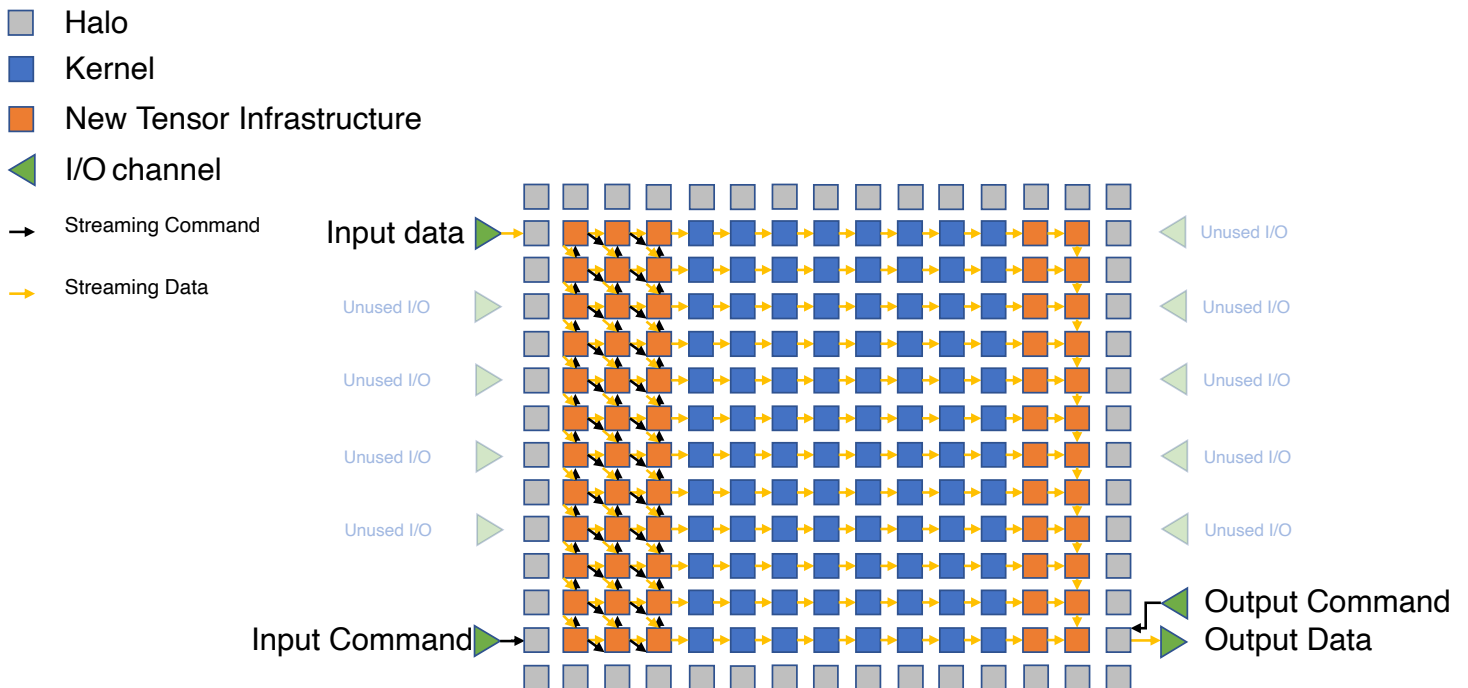
The Cerebras SDK provides a host runtime known as `SdkRuntime`, and associated functionality in the CSL `memcpy` library, to load programs, launch functions, and transfer data on and off the Wafer-Scale Engine.

The functions provided by `SdkRuntime` manage the data transfer to and from the host's filesystem or memory, through the host and WSE network interfaces, and finally route the data into your kernel. This last step is implemented on the WSE itself in order to connect the I/O channel entry-points, which are in fixed locations at the edges of the WSE, to each kernel, which has a variable size and location. These I/O channels are connected to the fabric routers at PEs on the East and West edges, spaced roughly 16 rows apart. On WSE-2, there are a total of 60 channels at each edge. On WSE-3, there are a total of 62 channels at each edge.



* I/O channels may not be placed at exactly 16 row intervals

The SDK **memcpy** infrastructure uses additional PEs around the user kernel to route tensor data and also adds a small executable component to the kernel PEs. In addition to a halo around the kernel, the additional support PEs consume three columns on the West of the kernel and two columns on the East.



SdkRuntime supports up to 16 I/O channels, and can further reduce the I/O latency by buffer insertion on either side of the core kernel.

Enabling Tensor Streaming

The `memcpy` infrastructure of `SdkRuntime` can either *stream* data into the device or copy data into a given device memory location directly. The former is called `streaming` mode and the latter is called `copy` mode. `streaming` mode requires the user to count the number of received wavelets in order to process the next task, while `copy` mode copies the data into the memory directly without notifying the user. Consider a tensor `A` and function `f` on the device which transforms `A`. To compute `f(A)` in the kernel, the user has two options:

1. `streaming` mode: Define a wavelet-triggered data task to receive the tensor `A` and call a function to compute `f(A)` after all of `A` is received.
2. `copy` mode: Copy the tensor `A` first, then launch a kernel function to compute `f(A)`.

To instantiate and use the `memcpy` infrastructure, you'll need to do the following:

- Pass the flags `--memcpy` and `--channels=k` to `cslc`, where `k` is an integer between 1 and 16, specifying the number of I/O channels to use.
- Specify `--fabric-dims=dim_x,dim_y` and `--fabric-offsets=x,y` to `cslc` such that `dim_x >= 7 + width`, `dim_y >= 2 + height`, `x >= 4`, and `y >= 1`, where `width` and `height` are the specified width and height of the program rectangle.
- Instantiate `memcpy` parameters in your top-level layout CSL file by importing `<memcpy/get_params>` with an `@import_module()` statement. This import will specify all needed parameters for the `memcpy` infrastructure, including `width` and `height` of the kernel, and any colors needed for `streaming` mode.
- Pass `memcpy` params to the PE program in the `@set_tile_code` call. Note that `memcpy` params are parameterized by the `x` coordinate of the PE in the program rectangle.
- Instantiate the `memcpy` module in your PE program by importing `<memcpy/memcpy>`.

Altogether, instantiating `memcpy` infrastructure in the top-level CSL file and the PE program will resemble the following example:

```
// in top-level CSL file
const memcpy = @import_module("<memcpy/get_params>", .{
    .width = width,
    .height = height
});

layout {
    @set_rectangle(1, 1);
    @set_tile_code(0, 0, "pe_program.csl", .{ .memcpy_params = memcpy.get_params(0) });
}

// in PE program CSL file pe_program.csl
param memcpy_params: comptime_struct;

const sys_mod = @import_module("<memcpy/memcpy>", memcpy_params);
```

Warning

The **memcpy** infrastructure uses colors 21, 22, 23, local task IDs 27, 28, 30, and control task IDs 33, 34, 35, 36, and 37. It also used microthread 0, input queue 0, and output queue 0. On WSE-3, **memcpy** additionally uses input queue 1. The compiler and runtime cannot detect all resource conflicts in your program. Do not use these resources in your program.

When using **streaming** mode, the user can block and unblock the input tensor colors. In particular, the user can overlap computation and communication by blocking/unblocking these colors.

The user must not set or modify the routing of an input, output tensor color or kernel launch color. The routing pattern is configured implicitly by the compiler. If the user modifies those routing patterns, the behavior is undefined.

Using Streaming Mode

To use **streaming** mode, the user must specify colors for host-to-device and device-to-host streaming. Input streaming colors are prefixed with **MEMCPYH2D_** and output streaming colors are prefixed with **MEMCPYD2H_**, followed by the tensor ID, an integer in the range 1-4. Unused colors should be omitted, and only four colors per direction are allowed.

Here's an example instantiation of a program in the top-level CSL file using colors for **memcpy** streaming:

```
// in top-level CSL file
// Compile-time IDs for memcpy streaming colors
param MEMCPYH2D_DATA_1_ID: i16;
param MEMCPYH2D_DATA_2_ID: i16;
param MEMCPYD2H_DATA_1_ID: i16;
param MEMCPYD2H_DATA_2_ID: i16;

// Generate colors from IDs
const MEMCPYH2D_DATA_1: color = @get_color(MEMCPYH2D_DATA_1_ID);
const MEMCPYH2D_DATA_2: color = @get_color(MEMCPYH2D_DATA_2_ID);
const MEMCPYD2H_DATA_1: color = @get_color(MEMCPYD2H_DATA_1_ID);
const MEMCPYD2H_DATA_2: color = @get_color(MEMCPYD2H_DATA_2_ID);

const memcpy = @import_module("<memcpy/get_params>", .{
    .width = width,
    .height = height,
    .MEMCPYH2D_1=MEMCPYH2D_DATA_1,
    .MEMCPYH2D_2=MEMCPYH2D_DATA_2,
    .MEMCPYD2H_1=MEMCPYD2H_DATA_1,
    .MEMCPYD2H_2=MEMCPYD2H_DATA_2
});
```

The user must also pass the input/output color ID and value pairs to **cs1c** as parameters, where **x** is the color ID:

```
--params=MEMCPYH2D_DATA_<x>_ID:<input_color>  
--params=MEMCPYD2H_DATA_<x>_ID:<output_color>
```

To stream the data into the device, the user can either use a data task to read the data from the input tensor color or use a microthread to read the data from a `fabin_dsd`. To bind a data task to an input color, call `@bind_data_task` at compile time:

```
const MEMCPYH2D_1_TASK_ID = @get_data_task_id(MEMCPYH2D_DATA_1);  
  
comptime {  
    // Task reads data on color MEMCPYH2D_DATA_1  
    @bind_data_task(memcpyh2d_data_1_task, MEMCPYH2D_DATA_1_TASK_ID);  
}
```

The user can send data to an output tensor color using a `fabout_dsd`. For instance:

```
@mov32(my_fabout_dsd, my_mem_buf_dsd, .{.async=true});
```

Using Copy Mode

To use `copy` mode to copy data to/from the device, the user has to define the symbols for the tensors to be copied.

For example, the following code defines a pointer `ptr_A` pointing to tensor `A`, and exports it.

```
// in top-level CSL file
const memcpy = @import_module("<memcpy/get_params>", .{
    .width = width,
    .height = height
});

layout {
    @set_rectangle(1, 1);
    @set_tile_code(0, 0, "pe_program.csl", .{ .memcpy_params = memcpy.get_params(0) });

    // export symbol names
    @export_name("A", [*]f32, true);
}

// in PE program CSL file pe_program.csl
param memcpy_params: comptime_struct;

const sys_mod = @import_module("<memcpy/memcpy>", memcpy_params);

var A = @zeros([4]f32);
var ptr_A : [*]f32 = &A;

comptime {
    @export_symbol(ptr_A, "A");
}
```

Launching Kernels

We can additionally use `memcpy` to launch a kernel function.

The following is an example of the kernel launching protocol. This program exports two functions to the host: `f1` and `f2`.

```

// in top-level CSL file
const memcpy = @import_module("<memcpy/get_params>", .{
    .width = width,
    .height = height
});

layout {
    @set_rectangle(1, 1);
    @set_tile_code(0, 0, "pe_program.csl", .{ .memcpy_params = memcpy.get_params(0) });

    // export symbol names
    @export_name("f1", fn()void);
    @export_name("f2", fn()void);
}

// in PE program CSL file pe_program.csl
param memcpy_params: comptime_struct;

const sys_mod = @import_module("<memcpy/memcpy>", memcpy_params);

fn f1() void {
    // do something
}

fn f2(my_arg: f32) void {
    // do something else
}

comptime {
    @export_symbol(f1);
    @export_symbol(f2);
}

```

Using Buffers

The compiler can insert buffers in the infrastructure to reduce the latency of the I/O. The buffer stores the wavelets from the I/O for one row of PEs while the core program rectangle is busy and cannot process the wavelets from the I/O. In other words, the buffer acts like a **prefetch** from the point of view of the computation.

There are two kind of buffers: one stores the data for host-to-device transfers, and the other stores the data for device-to-host transfers. The width of the former is configured by **--width-west-buf**, and the width of the latter is configured by **--width-east-buf**. By default, **--width-west-buf=0** and **--width-east-buf=0**, i.e., no buffers are inserted.

--width-west-buf=k means **k** columns of PEs are inserted to the West of the core kernel, and each PE can buffer 46 KB of data. If the user has 500 PEs in a row, then 46 KB can buffer 23 wavelets per PE (recall that each wavelet holds 32 bits of data). If the user wants to stream or copy a tensor of size 100 per PE, then **--width-west-buf=5** can buffer the whole tensor.

When compiling with `--width-west-buf=k` and `--width-east-buf=p`, the user must specify `--fabric-offsets=x,y` such that $x \geq 4 + k$ and $y \geq 1$, and `--fabric-dims=dim_x,dim_y` such that $\text{dim_x} \geq x + \text{width} + 3 + p$ and $\text{dim_y} \geq y + \text{height} + 1$, where `width` and `height` are the width and height of the program rectangle.

SdkRuntime Host API

See [SdkRuntime API Reference](#) for full documentation of the `SdkRuntime` Python host API.

The `SdkRuntime` Python host API supports memory transfers and kernel launches through the functions `memcpy_h2d()`, `memcpy_d2h()` and `launch()`: `memcpy_h2d()` is used for host-to-device data transfers, `memcpy_d2h()` is used for device-to-host data transfers, and `launch()` is used for kernel launches.

Each function can be a blocking or nonblocking call, depending on the parameter `nonblock` of the API. If blocking mode (`nonblock=False`) is selected, the API waits until the operation is done. Otherwise, the function returns before the operation even starts. `SdkRuntime` can aggregate multiple nonblocking operations together to reduce the latency. However the user must take care to avoid race conditions in nonblocking mode. For example, if the user has two `memcpy_d2h()` to the same destination, the content of the destination is undefined if both operations are nonblocking.

Instantiating SdkRuntime

You'll need to import the `SdkRuntime` module, as well as the `MemcpyDataType` and `MemcpyOrder` modules for specifying data type and ordering of tensors.

To create an `SdkRuntime` object, pass the directory which contains the ELF files produced by the compiler, and the IP address of the WSE, if running on hardware, to `SdkRuntime()`. The user can load the ELF files by `load()` and start the simulator or WSE with `run()`. After that, the user can do any operation, either memory transfers or kernel launches. Finally, the user calls `stop()` to shutdown the simulator or WSE.

```
from cerebras.sdk.runtime.sdkruntimepybind import SdkRuntime
from cerebras.sdk.runtime.sdkruntimepybind import MemcpyDataType
from cerebras.sdk.runtime.sdkruntimepybind import MemcpyOrder

simulator = SdkRuntime(args.name, cmaddr=args.cmaddr)
simulator.load()
simulator.run()
# a sequence of operations
simulator.stop()
```

Warning

Instantiating the `SdkRuntime` object uses slightly different syntax if you are compiling and running an SDK program on a Wafer-Scale Cluster in appliance mode. See [Running SDK on a Wafer-Scale Cluster](#).

Using `memcpy_h2d()` and `memcpy_d2h()`

The function `memcpy_h2d()` transfers a tensor from host to device using either `streaming` mode or `copy` mode. The function has the following parameters:

- `streaming=True` corresponds to `streaming` mode, and `streaming=False` to copy mode.
- The region of interest, or ROI, is a 4-tuple `(x, y, w, h)`, which indicates a subrectangle starting at `(x,y)` with (width, height) = `(w, h)`. The origin `(0, 0)` corresponds to the top left PE in your program rectangle; in absolute coordinates, this corresponds to the PE at the coordinates specified by `--fabric-offsets`. The ROI must lie within the program rectangle.
- The parameter `l` indicates number of elements (wavelets) per PE.
- The parameter `data_type` specifies either 16-bit (`MemcpyDataType.MEMCPY_16BIT`) or 32-bit (`MemcpyDataType.MEMCPY_32BIT`) for `copy` mode.
- The parameter `order` specifies row-major order (`MemcpyOrder.ROW_MAJOR`) or column-major order (`MemcpyOrder.COL_MAJOR`) for the input/output tensor of the form `A[h][w][l]`.
- The parameter `nonblock` indicates if the operation is blocking or nonblocking.
- The parameter `dest` is the color associated with this host-to-device transfer if `streaming=True` and is the symbol if `streaming=False`.

```
memcpy_h2d(dest, src, x, y, w, h, l, streaming, data_type, order, nonblock)
```

Similarly, the function `memcpy_d2h()` transfers a tensor from device to host using either `streaming` mode or `copy` mode. The first parameter `dest` is the host tensor to receive the data from the device. The second parameter `src` is the color associated with this device-to-host transfer if `streaming=True` or the device symbol from which to copy if `streaming=False`. All other parameters are the same as `memcpy_h2d()`.

```
memcpy_d2h(dest, src, x, y, w, h, l, streaming, data_type, order, nonblock)
```

The parameter `order` of `memcpy_h2d()` and `memcpy_d2h()` specifies either row-major or column-major. The host tensor from which or to which data is copied is a 1D array of length `w*h*l`, where `w` and `h` are the width and height of the region of interest in which to copy, and `l` is the number of elements per PE to copy. Row-major simply means that when mapping from 1D to `[w][h][l]`, `l` is the fastest varying dimension. Thus, elements contiguous on a PE will be contiguous on the host. For column-major, `w` is the fastest varying dimension. Note that the column-major version delivers better bandwidth than row-major.

`memcpy_h2d()` and `memcpy_d2h()` supports both 16-bit and 32-bit data transfer via `copy` mode or `streaming` mode. When using `memcpy_h2d()` for a 16-bit tensor, zero extension from 16-bit to 32-bit must be performed. When using `memcpy_d2h()` for a 16-bit tensor, the returned array will contain 32-bit data where the higher 16-bits are zero. The user has to strip out the higher 16-bits. See the [sdk_utils module documentation](#) for utilities to help perform this data transformation.

Using launch()

The `launch()` function is used for remote kernel launches of host-callable functions. The parameters are as follows:

- The first parameter `sym` is the symbol of the host-callable function.

- The next parameters are positional arguments, matching the arguments of the host-callable function.
- The last parameter `nonblock` is a keyword argument specifying whether the kernel launch is performed in blocking or nonblocking mode.

For example, to launch a host-callable function `my_fun` with two arguments of type `f32` in blocking mode, the call would look as follows:

```
my_fun_symbol = runner.get_symbol('my_fun')
runner.launch(my_fun_symbol, 1.0, 2.0, nonblock=False)
```